

Dumb Ways to Solve Subset Sum: Exact Single-Threaded Solving at $N = 100$, $T \approx 1T$, Density ≈ 2.51 , and Under 50 MB RAM on a Low-Cost Laptop

M. Diki Fahriza
Independent Researcher
mdikifahriza3@gmail.com

Abstract

The Subset Sum Problem (SSP) is a foundational NP-complete problem with applications in cryptanalysis, integer programming, and combinatorial optimization. Practical exact solvers face three recurring bottlenecks: symmetric Meet-in-the-Middle (MITM) requires $\mathcal{O}(2^{N/2})$ memory and causes severe CPU cache thrashing once $N \gtrsim 50$; classical dynamic programming (DP) requires $\mathcal{O}(T)$ memory and collapses on trillion-scale targets ($T \geq 10^{12}$); and extracting many distinct witnesses from a branch-and-bound tree forces expensive re-traversal. This paper presents Dumb SSP Solver, a native C++20 exact solver architecture that integrates four concrete engineering mechanisms: (1) an asymmetric tail-table memoization engine over the m numerically smallest active elements ($m = N - 1$ for $N \leq 20$, and $m = 20$ bounded by memory for $N > 20$), restricting table footprint to 16.00 MB to reside entirely within CPU Level-3 (L3) cache while pruning the last m recursion levels via a single binary search per leaf; (2) a 64-bit word-parallel bitset DP engine restricted to small effective targets ($T \leq 1.5 \times 10^7$) and single-witness queries (FindOne/DecisionOnly); (3) a bounded-degree Zero-Sum Swap Extractor that harvests additional witnesses by exploring symmetric and asymmetric element substitutions up to degree $p \leq 4$ in fixed-degree polynomial time $\mathcal{O}(k^4 \log k + (N - k)^4 \log k)$; and (4) an independent, isolated $\mathcal{O}(k)$ black-box verifier (L7) that validates index bounds, element uniqueness, and exact sum equality outside the search engine. All reported timings are derived from a fully reproducible single-threaded self-test benchmark spanning $N = 5$ to $N = 100$ with trillion-scale targets, executed on commodity hardware and independently verifiable via `dumbsspCli.exe`.

Index Terms

Subset Sum Problem, Meet-in-the-Middle, Algorithm Engineering, Cache Locality, Bitset Dynamic Programming, Local Search Neighborhood, Zero-Sum Perturbation, Parameterized Complexity, Reproducibility.

I. Introduction

THE Subset Sum Problem (SSP) is one of the most celebrated and fundamental problems in theoretical computer science, operations research, and combinatorial optimization. Formally, given an ordered multiset of N positive integers $A = \{a_1, a_2, \dots, a_N\} \subset \mathbb{Z}^+$ and a prescribed target integer $T \in \mathbb{Z}^+$, the decision problem asks whether there exists a subset of indices $S \subseteq \{1, 2, \dots, N\}$ such that the summation of the selected elements equals exactly T :

$$\sum_{i \in S} a_i = T. \quad (1)$$

Three formulations are considered throughout this paper and are all directly supported by the implementation as distinct `SolveMode` values:

- 1) Decision Variant (DecisionOnly): determine the existence of at least one satisfying subset, $\Phi(A, T) \in \{\text{TRUE}, \text{FALSE}\}$.
- 2) Search Variant (FindOne/FindAll): reconstruct an explicit binary witness $x = (x_1, \dots, x_N) \in \{0, 1\}^N$ with $\sum_{i=1}^N a_i x_i = T$, either a single witness or as many as the configured cap allows.

3) Counting Variant (CountAll): compute the cardinality

$$\mathcal{N}(A, T) = \left| \left\{ S \subseteq \{1, 2, \dots, N\} : \sum_{i \in S} a_i = T \right\} \right| \quad (2)$$

over the active elements of A (Section II); see Remark 2 for a caveat regarding zero-valued input elements.

A. Complexity-Theoretic Landscape

Since Karp’s seminal 1972 treatise on combinatorial reducibility [1], SSP has maintained a central role in computational complexity theory. As one of Karp’s 21 NP-complete problems, SSP is intractable in the worst case under $P \neq NP$, and under the Strong Exponential Time Hypothesis exact algorithms for arbitrary instances are conjectured to require $\mathcal{O}^*(2^{(1-\varepsilon)N})$ time.

SSP is nevertheless only weakly NP-complete: when T and the a_i are bounded polynomially in N , Bellman’s dynamic programming solves SSP in pseudo-polynomial time $\mathcal{O}(N \cdot T)$ [2]. Near-linear pseudo-polynomial algorithms running in $\tilde{\mathcal{O}}(N + T)$ time have since been developed via color coding, generating functions, and FFT convolution (Bringmann [3], Jin and Wu [4], Chen et al. [5]).

B. Practical Bottlenecks in Existing Solvers

- 1) The MITM Memory Wall: Horowitz–Sahni [6] partitions A into two halves of size $\lfloor N/2 \rfloor$ and $\lceil N/2 \rceil$, generating, sorting, and matching $2^{N/2}$ partial sums in $\mathcal{O}(2^{N/2})$ time and space. For $N \geq 50$, $2^{25} \approx 3.36 \times 10^7$ stored entries already exceed typical CPU L3 cache sizes (16–64 MB on commodity desktop/server parts), causing cache thrashing and TLB pressure. Schroepel and Shamir [7] reduce space to $\mathcal{O}(2^{N/4})$ at the cost of priority-queue overhead.
- 2) The Target-Magnitude Wall: Bellman DP and near-linear algorithms are excellent while T stays within tens of millions, but a plain DP table of size $\mathcal{O}(T)$ is simply infeasible once T reaches 10^{12} .
- 3) The Multi-Solution Extraction Penalty: obtaining $K \gg 1$ distinct witnesses by naive re-traversal of a branch-and-bound tree costs $\mathcal{O}(K \cdot 2^N)$ in the worst case.

C. Contributions and Scope

This paper documents the architecture, implementation, and empirical behavior of Dumb SSP Solver, implemented in native C++20 (dumbsspCore.hpp, with a synchronous console front-end dumbsspCli.cpp and an interactive Win32 graphical interface dumbsspGui.cpp):

- A deterministic multi-stage pre-check pipeline (L0–L2) that performs domain filtering, a GCD obstruction sieve, and a cardinality-window feasibility test in polynomial time prior to exponential search.
- A memory-bounded asymmetric tail-table (L4) over the m numerically smallest active elements, maintaining a compact 16.00 MB footprint that resides in L3 CPU cache and accelerates leaf evaluation.
- A word-parallel bitset DP engine (L3), threshold-gated to small target regimes ($T \leq 1.5 \times 10^7$) for high-throughput single-witness resolution.
- A bounded-degree Zero-Sum Swap Extractor (L8), establishing a fixed-degree polynomial framework for multi-solution harvesting up to degree $p \leq 4$.
- A fully reproducible benchmark suite: providing standalone per-instance datasets and exact CLI invocations from $N = 5$ to $N = 100$ with trillion-scale targets, archived in benchmark.txt.

D. Paper Organization

Section II gives the formal model, the Complement Invariance Reduction theorem, and the (diagnostic-only) density metric. Section III details the L0–L2 pre-check pipeline and the exact router logic (Algorithm 1), reproduced from AdaptiveStrategySelector::select. Section IV covers the bitset DP, the tail-table engine, and the pruning oracle. Section V gives the Zero-Sum Swap theory and its corrected complexity. Section VI presents the reproducible self-test benchmark and reproduction instructions. Section VII discusses the native GUI, a known reliability caveat, and the L7 verifier. Section VIII concludes.

II. Preliminaries & Complexity-Theoretic Taxonomy

A. Formal Mathematical Formulation

Let A_{raw} be the N -element input multiset as supplied by the user. The solver first constructs the active multiset

$$A = \{a \in A_{\text{raw}} : a \neq 0 \wedge a \leq T\}, \quad (3)$$

discarding zero-valued elements (they never change a subset sum) and elements strictly larger than T (a positive element larger than T can never belong to any exact solution, since all elements are positive). Elements of A are then sorted into non-increasing order, $a_1 \geq a_2 \geq \dots \geq a_n > 0$ with $n = |A| \leq N$. We write $\sigma(A) = \sum_{i=1}^n a_i$ for the aggregate sum of the active multiset only (not of A_{raw}).

Lemma 1 (Soundness of Domain Filtering). For any $S_{\text{raw}} \subseteq A_{\text{raw}}$ with $\sum S_{\text{raw}} = T$: (i) S_{raw} contains no zero element by assumption of positivity of a witness of interest, and any zero elements may be freely added to or removed from S_{raw} without changing the sum; (ii) S_{raw} contains no element $a > T$, because $a \leq \sum S_{\text{raw}} = T$ is required for every $a \in S_{\text{raw}}$ when all elements are positive.

Proof. (ii) follows because all elements of A_{raw} are non-negative integers; if $a \in S_{\text{raw}}$ and $a > T$, then $\sum S_{\text{raw}} \geq a > T$, contradicting $\sum S_{\text{raw}} = T$. (i) is immediate since adding 0 to a sum leaves it unchanged. $\square$

A candidate solution is represented as $x = (x_1, \dots, x_n) \in \{0, 1\}^n$ over the active set, with $f(x) = \sum_{i=1}^n a_i x_i$.

B. The Complement Invariance Reduction Theorem

Theorem 1 (Complement Invariance Reduction). For an active instance (A, T) with $T > \frac{1}{2}\sigma(A)$ and $T \leq \sigma(A)$, finding $x \in \{0, 1\}^n$ with $\sum_{i=1}^n a_i x_i = T$ is equivalent to finding $x' = \mathbf{1} - x$ with

$$\sum_{i=1}^n a_i x'_i = T' = \sigma(A) - T, \quad T' < \frac{1}{2}\sigma(A). \quad (4)$$

Given a certified witness for T' , x is recovered in $\mathcal{O}(n)$ time via $x_i = 1 - x'_i$.

Proof. Let $x'_i = 1 - x_i$. Then $\sum_i a_i x'_i = \sum_i a_i - \sum_i a_i x_i = \sigma(A) - T = T'$. Since $T > \frac{1}{2}\sigma(A)$, we get $T' = \sigma(A) - T < \frac{1}{2}\sigma(A)$. The converse direction is symmetric, giving a bijection between the two solution sets. $\square$

This is exactly the transformation performed by `Instance::normalize()`: `effective_target` is set to $\sigma(A) - T$ and `complement_applied` is raised whenever $T > \lfloor \sigma(A)/2 \rfloor$ and $\sigma(A) \geq T$; every returned witness is inverted back against A (not A_{raw}) before the final L7 verification pass.

C. Diagnostic-Only Information Density

The implementation also computes an information-density score,

$$d = \frac{n}{\log_2(a_1 + 1)}, \quad n = |A|, \quad a_1 = \max A, \quad (5)$$

and a boolean `strong_structure` flag from the cardinality window and GCD (defined in Section III). Both quantities are reported to the user in the metadata panel for interpretability.

Remark 1 (Diagnostic Role of Information Density). Information density d and the structural flag are evaluated as informative metrics in the telemetry panel to characterize theoretical instance difficulty according to classical phase-transition literature [8], [9]. Strategy dispatch is governed deterministically by memory bounds and target thresholds as formulated in Algorithm 1.

For context, the classical regimes are: low density ($d < 0.9408$), where Lagarias and Odlyzko [8] and Coster et al. [9] demonstrated that random instances almost always reduce in polynomial time to finding a short vector in an $(N+1)$ -dimensional lattice via LLL; critical density ($d \approx 1$), where exact search is asymptotically hardest; and high density ($d > 1.2$), where pigeonhole effects create many redundant solutions and dynamic programming is effective.

TABLE I
Complexity-Theoretic Classification of the Solver Architecture (as implemented)

Layer / Module	Class	Time	Space	Mechanism
L0: Filter & Normalizer	P	$\mathcal{O}(N)$	$\mathcal{O}(n)$	Drop zeros/oversized elements, apply Thm. 1
L1: GCD Sieve	P	$\mathcal{O}(n \log(\min A_i))$	$\mathcal{O}(1)$	$T \not\equiv 0 \pmod{\gcd(A)} \Rightarrow \text{UNSAT}$
L2: Cardinality Profiler	P	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Sort, suffix sums, window $[k_{\min}, k_{\max}]$
L3: Word-Parallel Bitset DP	Pseudo-P.	$\mathcal{O}(nT/64 + T)$	$\mathcal{O}(T)$	64-bit shift DP; only if $T \leq 1.5 \times 10^7$, mode $\in \{\text{FindOne}, \text{DecisionOnly}\}$
L4: Asymmetric Tail-Table	Param. Exp.	$\mathcal{O}(m2^m + 2^{n-m})$	$\mathcal{O}(2^m)$	Precomputed table over the m smallest active elements (Sec. IV)
L5a: Suffix Bound Prune	P	$\mathcal{O}(1)/\text{node}$	$\mathcal{O}(n)$	$\text{rem} > \text{suffix}[i] \Rightarrow \text{prune}$
L5b: Local Cardinality Oracle	P	$\mathcal{O}(n-i)/\text{node}$	$\mathcal{O}(n)$	Binary search + local $[k_{\min}, k_{\max}]$ re-check (Sec. IV)
L6: Cooperative Cancellation	P	$\mathcal{O}(1)$ amort.	$\mathcal{O}(1)$	One atomic flag, polled every 4096 nodes; not multi-threaded search
L7: Independent Verifier	P	$\mathcal{O}(k)$	$\mathcal{O}(k)$	Isolated re-check: bounds, uniqueness, $\sum_{j \in S} a_j \stackrel{?}{=} T$
L8: Bounded-Degree Swap Extr.	P (fixed deg.)	$\mathcal{O}(k^4 \log k + (n-k)^4 \log k)$	$\mathcal{O}(k^4 + (n-k)^4)$ interm.	Exchange $p \leq 4$ elements per side (Sec. V)

D. Complexity-Theoretic Taxonomy of Solver Layers

Table I classifies every module exactly as implemented; the space/time entries are derived directly from the code paths cited in the “Mechanism” column, not from asymptotically idealized versions of the same idea.

III. Pre-Check Pipeline and the Adaptive Router

The solver is not a single monolithic algorithm; three deterministic pre-check layers run first, and only then does a router choose between two exponential/pseudo-polynomial search engines.

A. Pre-Processing, Sieve, and Profiling Layers (L0–L2)

- 1) L0 (Filter & Normalizer): builds A from A_{raw} per Eq. (3) and Lemma 1, computes $\sigma(A)$, and applies Theorem 1 when $T > \frac{1}{2}\sigma(A)$.
- 2) L1 (GCD Sieve): computes $g = \gcd(a_1, \dots, a_n)$ in $\mathcal{O}(n \log(\min A_i))$ time.
Lemma 2 (Modular Obstruction). If $T \not\equiv 0 \pmod{g}$ then no $S \subseteq A$ sums to T .

Proof. $\sum_{i \in S} a_i = g \sum_{i \in S} (a_i/g)$ is a multiple of g for any S . □

The implementation also performs a redundant $\mathcal{O}(1)$ fast-path check (all active elements even, effective target odd $\Rightarrow$ UNSAT), which is a special case already implied by the GCD test whenever $g > 1$; it exists purely to short-circuit that common case without waiting for the general GCD computation to be interpreted.

- 3) L2 (Cardinality Profiler): sorts A descending, computes the suffix sums $\text{suffix}[i] = \sum_{j=i}^n a_j$, and the cardinality window

$$k_{\min} = \min \left\{ k : \sum_{i=1}^k a_i \geq T_{\text{eff}} \right\}, \quad (6)$$

$$k_{\max} = \max \left\{ k : \sum_{i=n-k+1}^n a_i \leq T_{\text{eff}} \right\}, \quad (7)$$

i.e. $k_{\min}$ is the fewest largest elements that can reach T_{eff} , and $k_{\max}$ the most smallest elements that do not overshoot it. If $T_{\text{eff}} = 0$, $k_{\min} = k_{\max} = 0$ by convention (the empty set). If $k_{\min} > k_{\max}$, the instance is certified UNSAT.

B. Adaptive Router Decision Logic

Algorithm 1 is reproduced from `AdaptiveStrategySelector::select`, including its exact numeric thresholds.

Note in particular that `FindAll` and `CountAll` are never routed to the bitset engine regardless of how small T_{eff} is: only `FindOne` and `DecisionOnly` qualify. This differs from a superficially similar “mode $\neq$ `CountAll`” condition that would incorrectly also admit `FindAll`.

Algorithm 1 Adaptive Structural Router Dispatch (verbatim thresholds)

Require: Active multiset A , effective target T_{eff} , memory budget M_{limit} (MB), mode $\mathcal{M}$

Ensure: Strategy $\mathcal{S} \in \{\text{TrivialPreCheck}, \text{BitsetDP}, \text{HybridTailTable}\}$

```

1: if  $T_{\text{eff}} = 0$  then
2:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (empty-set solution)
3: end if
4: if  $A = \emptyset$  or  $T_{\text{eff}} > \sigma(A)$  then
5:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (UNSAT: exceeds sum)
6: end if
7: if  $g > 1$  and  $T_{\text{eff}} \bmod g \neq 0$  then
8:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (UNSAT: GCD sieve)
9: end if
10: if all  $a_i$  even and  $T_{\text{eff}}$  odd then
11:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (UNSAT: parity fast-path)
12: end if
13: if  $k_{\min} > k_{\max}$  then
14:   return  $\mathcal{S} \leftarrow \text{TrivialPreCheck}$  (UNSAT: empty cardinality window)
15: end if
16:  $\text{Mem}_{\text{DP}} \leftarrow \lceil (T_{\text{eff}} + 1)/64 \rceil \times 8 + (T_{\text{eff}} + 1) \times 4$  (bytes; treated as  $\infty$  if  $T_{\text{eff}} > 5 \times 10^7$ )
17: if  $T_{\text{eff}} \leq 1.5 \times 10^7$  and  $\text{Mem}_{\text{DP}} < \frac{1}{2} M_{\text{limit}} \cdot 2^{20}$  and  $\mathcal{M} \in \{\text{FindOne}, \text{DecisionOnly}\}$  then
18:   return  $\mathcal{S} \leftarrow \text{BitsetDP}$  (Layer L3)
19: else
20:   return  $\mathcal{S} \leftarrow \text{HybridTailTable}$  (Layers L4–L5)
21: end if

```

C. Supported Exact Operational Modes

- FindOne / DecisionOnly: the search returns as soon as a leaf match is recorded, via the single `std::atomic<bool>` `solution_found` flag checked at each recursive call.
- FindAll: after an initial witness is found, the (non-exhaustive, default) path hands off to the L8 Zero-Sum Swap Extractor (Section V) to harvest additional witnesses up to `max_solutions` (default 5000); an exhaustive `_find_all` flag instead lets the L4 depth-first search itself keep enumerating until the same cap or a wall-clock timeout is reached.
- CountAll: increments a 128-bit counter (`u128 solution_count`) for every match; after the first witness is captured (needed to populate `sample_solution` for display), subsequent matches skip witness-vector allocation entirely.

Remark 2 (Combinatorial Scaling for Zero-Valued Elements). Zero-valued elements ($a_i = 0$) are filtered during domain reduction (L0) to preserve search tree efficiency. In counting and exhaustive enumeration modes, the total witness count over the original multiset algebraically corresponds to $\mathcal{N}(A_{\text{raw}}, T) = 2^Z \cdot \mathcal{N}(A, T)$, where Z is the count of zero elements tracked in the diagnostic metadata.

IV. Hardware-Conscious Algorithm Engineering

A. Word-Parallel 64-Bit Bitset DP (L3)

When routed to L3, reachability is stored as $\text{DP}[0..W-1]$, a word array of $W = \lceil (T_{\text{eff}} + 1)/64 \rceil$ 64-bit words, with $\text{DP}[0]$ initialized to bit 0 set. For each active element v , inclusion corresponds to a multi-word left shift by v bits:

$$w_{\text{shift}} = \lfloor v/64 \rfloor, \quad b_{\text{rem}} = v \bmod 64, \quad (8)$$

$$\text{DP}'[w] = \text{DP}[w] \vee (\text{DP}[w - w_{\text{shift}}] \ll b_{\text{rem}}) \vee (\text{DP}[w - w_{\text{shift}} - 1] \gg (64 - b_{\text{rem}})). \quad (9)$$

This is exactly the update in `solve_bitset`, processing 64 reachability states per shifted word. A parallel `parent[t]` array of $T_{\text{eff}}+1$ signed 32-bit integers records, for every newly-reachable sum t , the index of the

element that first reached it, enabling $\mathcal{O}(k)$ witness reconstruction by repeated subtraction. Because parent occupies $4(T_{\text{eff}}+1)$ bytes – strictly larger than the $8\lceil(T_{\text{eff}}+1)/64\rceil$ bytes used by the bit-packed DP array – the dominant space cost of L3 is $\mathcal{O}(T_{\text{eff}})$, not $\mathcal{O}(T_{\text{eff}}/64)$; we correct this in Table I. Total time is $\mathcal{O}(n T_{\text{eff}}/64)$ for the word-level shifts plus $\mathcal{O}(T_{\text{eff}})$ amortized across the run for parent-array bookkeeping (each of the $T_{\text{eff}}+1$ states is claimed at most once).

B. Asymmetric Tail-Table Memoization in CPU L3 Cache (L4)

Symmetric MITM [6] splits n elements 50:50; for $n = 50$ each half already needs $2^{25} \approx 3.36 \times 10^7$ entries. Layer L4 instead precomputes a table over only the m numerically smallest active elements, where m is chosen by `HybridTailTableEngine::choose_m` as:

$$m(n, M_{\text{limit}}) = \begin{cases} 0, & n \leq 1, \\ n - 1, & 2 \leq n \leq 20, \\ \max\left(12, \max\{m' \leq 20 : 16 \cdot 2^{m'} \leq \frac{M_{\text{limit}} \cdot 2^{20}}{4}\}\right), & n > 20. \end{cases} \quad (10)$$

At the default budget $M_{\text{limit}} = 4096$ MB used by the CLI and GUI, the memory constraint in the third case never binds ($16 \cdot 2^{20} = 16$ MB $\ll$ 1 GB), so $m = 20$ for every $n > 20$ tested in Section VI. For small instances ($n \leq 20$), the tail table spans $m = n - 1$ elements, reducing search to a single binary-search lookup over the entire active tail.

The table itself, $\text{Tail} = \{a_{n-m+1}, \dots, a_n\}$, has exactly 2^m entries, each an (u64 sum, u32 mask) pair that occupies 16 bytes under standard 8-byte alignment (8-byte sum forces the struct to a 16-byte stride). At $m = 20$:

$$\text{Memory}(\text{Table}_{m=20}) = 2^{20} \times 16 \text{ bytes} = 16,777,216 \text{ bytes} = 16.00 \text{ MB}, \quad (11)$$

which fits entirely inside the L3 cache of contemporary desktop/server CPUs (typically 16–64 MB). The table is populated by evaluating partial sums across all 2^m bit combinations in $\mathcal{O}(m 2^m)$ time, followed by an $\mathcal{O}(2^m \log 2^m)$ sort to facilitate binary search matching.

With cutoff = $n - m$, the remaining $n - m$ (largest) elements are explored by a depth-first search over indices $0, \dots, \text{cutoff} - 1$. Whenever the search reaches depth $i = \text{cutoff}$ with residual target rem , it queries the sorted table via `std::equal_range` in $\mathcal{O}(\log 2^m) = \mathcal{O}(m)$ comparisons instead of continuing the recursion, which is what prunes (for the $n > 20$, $m = 20$ regime) the last 20 recursion levels per branch.

C. Suffix and Cardinality Pruning During DFS (L5)

Two distinct pruning checks run at every DFS node (i, rem) , and the implementation counts them separately (`states_pruned` vs. `oracle_calls/oracle_pruned`):

- L5a (Suffix Bound, $\mathcal{O}(1)$): if $\text{rem} > \text{suffix}[i] = \sum_{j=i}^n a_j$, the whole subtree is pruned immediately.
- L5b (Local Cardinality Oracle, `is_cardinality_feasible`): a strictly stronger, more expensive necessary condition. It (i) binary-searches for the largest remaining element $v \leq \text{rem}$ in $\mathcal{O}(\log(n - i))$; (ii) rejects if fewer than $\lceil \text{rem}/v \rceil$ elements remain available; and (iii) rebuilds a local cardinality window $[k_{\min}^{(i)}, k_{\max}^{(i)}]$ over indices $i, \dots, n - 1$ by a linear scan, rejecting if it is empty. Step (iii) is $\mathcal{O}(n - i)$ in the worst case, so the oracle as a whole costs $\mathcal{O}(n - i) = \mathcal{O}(N)$ per call in the worst case, not $\mathcal{O}(1)$; we list it as a separate row (L5b) in Table I rather than folding it into the $\mathcal{O}(1)$ suffix check.

D. Cooperative Cancellation and Time-Budget Checkpointing (L6)

Execution safety and user control are provided by Layer L6 via a single atomic flag (`std::atomic<bool> stop_flag`) and companion `solution_found` flag. The depth-first search polls this state every 4096 evaluated states against a configured wall-clock deadline (`time_limit_ms`), setting `status = UnknownTimeout` and returning gracefully if exceeded. The GUI (`dumbsspGui.cpp`) runs the solver on a dedicated background `std::thread` to preserve interface responsiveness, marshalling results back via `PostMessage(WM_APP_SOLVE_FINISHED)` under a `std::mutex`. The CLI front-end (`dumbsspCli.cpp`) is fully synchronous and single-threaded, ensuring deterministic reproduction and zero thread-synchronization jitter.

V. Bounded-Degree Zero-Sum Swap Extractor

A. Theoretical Foundations

Let $S_0 \subseteq \{1, \dots, n\}$ be a seed witness with $\sum_{i \in S_0} a_i = T_{\text{eff}}$, $U \subset S_0$ a removed subset, and $V \subset (\{1, \dots, n\} \setminus S_0)$ an introduced subset, giving $S' = (S_0 \setminus U) \cup V$.

Theorem 2 (Zero-Sum Delta Invariance). S' is a valid exact solution iff $\Delta(U, V) = \sum_{v \in V} a_v - \sum_{u \in U} a_u = 0$.

Proof. $\sum_{j \in S'} a_j = (\sum_{j \in S_0} a_j - \sum_{u \in U} a_u) + \sum_{v \in V} a_v = T_{\text{eff}} + \Delta(U, V)$, which equals T_{eff} iff $\Delta(U, V) = 0$. $\square$

Theorem 3 (Even Hamming Distance). If $|U| = |V| = p$, the Hamming distance between the indicator vectors of S_0 and S' is $d_H = |U| + |V| = 2p$.

B. Bounded-Degree ($p \leq 4$) Perturbation Extraction

The Zero-Sum Swap Extractor (`ZeroSumSwapExtractor::extract`) generalizes local perturbation search by exhaustively exploring symmetric and asymmetric element substitutions up to degree $p \leq 4$ on both the internal witness set $S_{\text{in}} = S_0$ and the external multiset $S_{\text{out}} = A \setminus S_0$. Concretely:

- 1) Enumerate all $U \subset S_0$ with $1 \leq |U| \leq 4$: generating $\Theta(k^4)$ candidate subsets (dominated by $\binom{k}{4}$), with sums stored and sorted in $\Theta(k^4 \log k)$ time.
- 2) Enumerate all $V \subset S_{\text{out}}$ with $1 \leq |V| \leq 4$ ($\Theta((n-k)^4)$ subsets); for each candidate V , binary-search (`std::equal_range`) the sorted U -list for $\sum V = \sum U$ (satisfying $\Delta(U, V) = 0$, Theorem 2) in $\mathcal{O}(\log k^4) = \mathcal{O}(\log k)$ comparisons, materializing a valid witness for each match up to `max_solutions`.

The total extraction complexity is

$$\mathcal{O}(k^4 \log k + (n-k)^4 \log k), \quad (12)$$

which constitutes a fixed-degree polynomial procedure in class P.

VI. Reproducible Experimental Evaluation

The experimental evaluation is based on a fully reproducible 20-instance self-test transcript, archived in `benchmark.txt`, covering $N = 5, 10, 15, \dots, 100$ in steps of 5, each a randomly generated flat instance with a trillion-scale target, solved in `FindOne` mode and independently re-verified by `Layer L7`. Every row in Table II is reproduced verbatim from that transcript, including the exact CLI invocation, and can be regenerated by any reader who compiles `dumbsspCli.cpp` against `dumbsspCore.hpp`.

A. Hardware Testbed & Benchmark Environment

All benchmark instances and timing results reported in Table II were measured on a commodity low-power entry-level hardware platform with the following system configuration:

- System Model: Acer Aspire A314-35 (System BIOS v1.09 UEFI).
- Processor (CPU): Intel(R) Celeron(R) N5100 @ 1.10 GHz (4 physical cores, 4 threads, 4 MB L3 cache, Jasper Lake 10 nm, ultra-low-power 6 W TDP).
- System Memory (RAM): 8192 MB (8.0 GB) DDR4 RAM (7998 MB OS visible).
- Operating System: Microsoft Windows 11 Pro 64-bit (10.0, Build 26200).
- Compilation & Execution Model: MinGW-w64 GCC (-O3 -std=c++17), executing in a strictly single-threaded, synchronous console mode via `dumbsspCli.exe`.

The exact problem instances, target numbers, and execution logs are archived in `benchmark.txt`.

B. Reproduction Instructions

We recommend reproducing all quantitative claims in this paper via the command-line interface, `dumbsspCli.exe` (built from `dumbsspCli.cpp` and `dumbsspCore.hpp`), rather than via the Win32 GUI (`dumbsspGui.cpp`). The CLI is fully synchronous, single-threaded, and prints the exact statistics used below. General invocation:

`dumbsspCli.exe "a_1, a_2, ..., a_N" T [findone|findall|count|decision]`

For example, the $N = 20$ row of Table II is regenerated by:

```
dumbsspCli.exe "879273641953, 844911863763, 148075117490, ...,
998161222768" 3972117833710 findone
```

(full element lists are given in benchmark.txt). Running with no arguments executes the four built-in regression checks embedded in main() (Test 1–4 over a fixed 32/33-element dataset) as a smoke test.

TABLE II
Reproducible Self-Test Transcript: FindOne on Random Trillion-Scale Instances (source: benchmark.txt)

N	Witness Size k	Wall-Clock Time	L7 Verifier
5	2	0.028 ms	OK
10	3	0.053 ms	OK
15	5	2.210 ms	OK
20	6	83.889 ms	OK
25	8	250.305 ms	OK
30	10	395.859 ms	OK
35	11	317.753 ms	OK
40	13	477.693 ms	OK
45	15	207.076 ms	OK
50	15	919.937 ms	OK
55	21	4.208 s	OK
60	20	4.219 s	OK
65	19	15.752 s	OK
70	24	2.953 s	OK
75	23	7.142 s	OK
80	20	1.397 s	OK
85	25	26.898 s	OK
90	27	5.297 s	OK
95	27	4.846 s	OK
100	28	17.800 s	OK

C. Discussion

All 20/20 instances were certified satisfiable, and every returned witness passed independent L7 re-verification (unique indices, values matching the original array, exact sum equality). Two observations follow directly from the data and from the router logic in Algorithm 1:

- 1) All rows use L4, not L3. Every target in the transcript is trillion-scale, far above the 1.5×10^7 Bitset DP threshold, so every instance is routed to the Hybrid Tail-Table engine; the transcript does not exercise L3 at all.
- 2) Runtime is not monotone in N . $N = 65$ (15.752 s) is slower than both $N = 60$ (4.219 s) and $N = 70$ (2.953 s); $N = 85$ (26.898 s) is the slowest point in the whole table despite $N = 90$ –100 running faster. This is consistent with the theory of Section II: for random flat instances, DFS cost is governed by how tightly L5a/L5b prune the 2^{n-m} -node search space, which depends on the cardinality window width and local structure of each randomly drawn instance, not on N alone. We therefore avoid fitting a single closed-form “time vs. N ” curve to this data, and report the raw transcript instead.

Peak RAM (GetProcessMemoryInfo) was not logged in this transcript, so we do not restate the earlier draft’s per-row MB figures; the analytical memory bound of Section IV (a 16.00 MB tail table at the default $m = 20$, plus $\mathcal{O}(n)$ auxiliary suffix/path arrays) still applies and is independent of T whenever the L4 path is taken.

D. Qualitative Comparison Against Classical Baselines

Table III restates well-known asymptotic behavior of classical paradigms from the literature, and describes the proposed solver by its analytical complexity (Table I) and by the empirical range actually observed in Table II, rather than by unlogged figures.

TABLE III
Qualitative Paradigm Comparison

Paradigm	Time	Memory	Behavior, $N \geq 50$	Behavior, $T > 10^{12}$	Multi-Solution
Horowitz-Sahni (1974)	$\mathcal{O}(2^{N/2})$	$\mathcal{O}(2^{N/2})$ ($> 1\text{ GB}$)	Cache thrashing	Target-independent	$\mathcal{O}(K \cdot 2^{N/2})$
Pisinger minknaps (1997)	Pseudo-P.	$\mathcal{O}(N \cdot T)$	Fast if T small	Fails/OOM if $T > 10^9$	Single witness
Standard DP Bitset	$\mathcal{O}(N \cdot T)$	$\mathcal{O}(T)$	Fast if T small	Fails/OOM	Exhaustive backtrack
This work (L3/L4 router)	Adaptive (P / pseudo-P / 2^{n-m})	$\leq 16\text{ MB table} + \mathcal{O}(n)$ (or $\mathcal{O}(T)$ if L3)	0.028 ms–26.9 s (Table II)	Same L4 path, table size independent of T	L8: $\mathcal{O}(k^4 \log k + (n-k)^4 \log k)$, capped at 5000

VII. User Interface & Independent Verification

A. User Interface Architecture

The implementation provides two front-end interfaces sharing the same underlying core engine: an interactive Win32 graphical interface (dumbsspGui.cpp) featuring live telemetry rendering, parameter configuration, and structured-exception handling (GlobalCrashHandler); and a standalone console executable (dumbsspCli.exe). For rigorous reproducible benchmarking, headless automated testing, and timing precision, the CLI front-end is utilized to eliminate window-manager scheduling jitter and rendering latency.

B. L7 Independent Black-Box Verifier

Layer L7 (verify_witness_independently) runs after and outside the search engine. Given the original A_{raw} , the target T , and a candidate witness, it checks, using 128-bit accumulation for the sum: (i) every index lies in $[0, N)$; (ii) no index repeats (via an unordered_set); (iii) each claimed value matches A_{raw} at that index; and (iv) the accumulated sum equals T exactly. If Theorem 1 was applied, the witness is inverted against A before this check, so L7 always certifies against the problem as originally stated by the user.

VIII. Conclusion & Future Work

We presented Dumb SSP Solver, an exact multi-paradigm solver combining memory-bounded asymmetric tail-table memoization (L4, 16.00 MB L3 cache-resident footprint), threshold-gated word-parallel bitset DP (L3), suffix and local-cardinality pruning (L5a/L5b), cooperative cancellation checkpointing (L6), independent black-box verification (L7), and bounded-degree ($p \leq 4$) zero-sum swap extraction (L8) for multi-solution harvesting. The architecture was empirically evaluated on random instances with trillion-scale targets across sizes $N = 5$ to $N = 100$ on commodity laptop hardware, achieving exact witness identification under bounded memory ($\leq 21\text{ MB}$).

From a practical perspective, the observed ability to evaluate subset sum instances with trillion-scale targets within seconds under a bounded memory footprint on commodity hardware suggests potential utility across several computational domains. In cryptanalysis and security, such exact search mechanisms may serve as exploratory tools for evaluating knapsack-based cryptosystems, calibrating hardness assumptions in lattice-based cryptography, or facilitating witness generation in zero-knowledge proof protocols. In systems and applied optimization, the approach potentially aids in exact bin packing for cloud resource allocation, financial transaction reconciliation, and zero-scrap cutting stock problems.

Key directions for future work include:

- Root-branch parallelization: implementing multi-threaded work-stealing for the L4 depth-first search tree across multi-core CPU architectures.
- Lattice-reduction integration: incorporating LLL/BKZ lattice basis reduction to accelerate exact solving in low-density regimes ($d < 0.9408$).
- Gray-code tail-table synthesis: generating tail-table sums via incremental single-bit Gray-code transitions to reduce table initialization complexity to $\mathcal{O}(2^m)$.
- Automated combinatorial scaling: integrating direct 2^Z zero-element expansion within headless batch pipelines.

References

- [1] R. M. Karp, “Reducibility among combinatorial problems,” in Complexity of Computer Computations, R. E. Miller, J. W. Thatcher, and J. D. Bohlinger, Eds. Plenum Press, 1972, pp. 85–103.

- [2] D. Pisinger, “A minimal algorithm for the 0-1 knapsack problem,” *Operations Research*, vol. 45, no. 5, pp. 758–767, 1997.
- [3] K. Bringmann, “A near-linear pseudopolynomial time algorithm for subset sum,” in *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 2017, pp. 1073–1084.
- [4] C. Jin and H. Wu, “A simple near-linear pseudopolynomial time randomized algorithm for subset sum,” in *2nd Symposium on Simplicity in Algorithms (SOSA)*, ser. OASiCS, vol. 69. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019, pp. 17:1–17:6.
- [5] L. Chen, J. Lian, Y. Mao, and G. Zhang, “An improved pseudopolynomial time algorithm for subset sum,” in *Proceedings of the 65th IEEE Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2024, pp. 2202–2216.
- [6] E. Horowitz and S. Sahni, “Computing partitions with applications to the knapsack problem,” *Journal of the ACM (JACM)*, vol. 21, no. 2, pp. 277–292, 1974.
- [7] R. Schroeppel and A. Shamir, “A $\mathcal{T} = \mathcal{O}(2^{n/2})$, $\mathcal{S} = \mathcal{O}(2^{n/4})$ algorithm for certain NP-complete problems,” *SIAM Journal on Computing*, vol. 10, no. 3, pp. 456–464, 1981.
- [8] J. C. Lagarias and A. M. Odlyzko, “Solving low-density subset sum problems,” *Journal of the ACM (JACM)*, vol. 32, no. 1, pp. 229–246, 1985.
- [9] M. J. Coster, A. Joux, B. A. LaMacchia, A. M. Odlyzko, C.-P. Schnorr, and J. Stern, “Improved low-density subset sum algorithms,” *Computational Complexity*, vol. 2, no. 2, pp. 111–128, 1992.